Heap sort

The data structure used for heap sort is a binary heap. A binary heap allows two important operations insert, and deletemin while preserving heap property. Heap property is explained as follows:

- Heap structure is a complete binary tree such that all except depthmost level has are completely filled.
- Depthmost level is filled from left to right. No node in be depthmost level can be right child of its parent unless it has a left sibling (see figure below).
- The element stored at every node is smaller than those stored at any of children nodes.

The first two properties deal with the shape of a heap, and allows us to store it in an one dimentional array such that the node at index i has its parent at $\lfloor i/2 \rfloor$. Equivalently, the node at index postion i has its children located at index positions $2i$ and $2 + 1$. The nodes are numbered in level order which makes it clear that the structure can be stored in an array as explained previously.

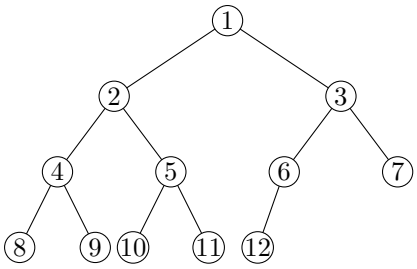
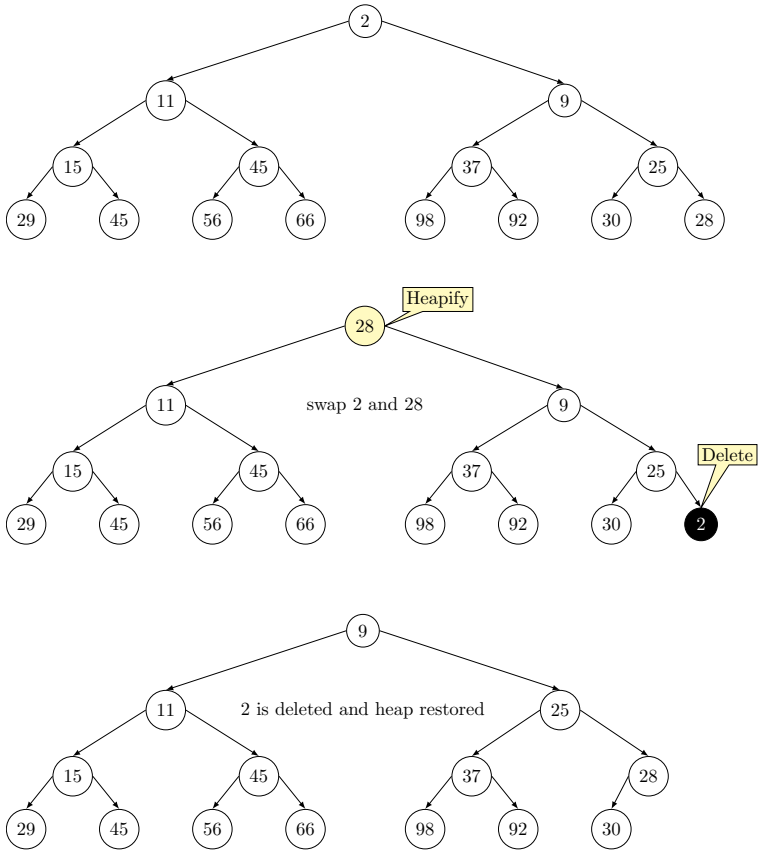


Figure below shows a heap structure and a deletemin operation on it. If we order the nodes in level order, then heap structure enables us to store a



Given an array, we assume the element at index 0 as the root and define the heap by letting elements at positions $2i$ and $2i + 1$ for $i = 1, 2, 3, \ldots$. Perforing heapify bottom up we restore the heap property.

We can build a heap incrementally from empty node and then use deletemin repeatedly to sort the given set of input elements. The process is very simple to understand needs no proof of correctness. All we need to ensure is that heap property is preserved while we perform any mutating operation such insert and deletemin with a heap. Since, insert and deletemin operations on a heap with n nodes can be performed in time of O($\log n$), heap sort has a running time of O($n \log n$).

The first thing we do is to create a heap from a given array. There are two options for it.

- Shiftup, and
- Shiftdown

Shiftup starts at the top of the heap, i.e., the beginning of the array, and calls shiftup on each item of the array. At the beginning of each iteration, we assume that the items before the current incoming item form a valid item. Incoming item is compared with the item already in the heap, and items are shifted up to create a position for the new item.

Shiftdown start from the bottom of the heap, i.e., the end of the array, and call shiftup with incoming item from backwards moving towards the front of the array. The items in heap are shiftdown to create a gap for the new item. Shiftdown is is more efficient than the shiftup.

Let us discuss the algorithm a bit to understand its logic.

- Start from the end of the array and create heap as execution progresses towards the beginning of the array.
- Since only one element pushed at time, the important strategy is to fix the heap property between the elements in heap and the incoming element.

Let us assume that array index start from 0 and ends at $n - 1$.

- The first non-leaf node in an array occurs at index position $n/2$.
- Build heap should start at $n/2$ and gradually work up to index 0.

Once index manipulation scheme is clear, we need to incorporate it into shiftdown operation. The children of the new node i would be at index positions $2i + 1$ and $2i+2$, for $i = 0, \ldots n/2$. It handles the issue of index starting from 0. We compare the incoming element of array at position i with element at $2i + 1$ and $2i + 2$ and choose the element to shiftdown which ever is smallest of the children and smaller element at i. It preserves the heap property that element at parent is smaller than the elements both at left child and right child. So the logic is simply as follows.

```
smallest = i
left = 2i+1
right = 2i+2

# Checks if left exists and smaller than current
if left < n and arr[left] < arr[smallest]:
    smallest = left

# Checks if right exists and smaller than current
```

```
if right < n and arr[right] < arr[smallest]:
    smallest = right
```

Let us breakdown shiftdown operations for creating a heap from an array with n elements.

- At most $n/2$ nodes are at leaf node positions, which implies $n/2$ nodes do not require any shiftdown.
- From the remaining $n/2$ nodes, at most half of them occur just one level up from the leaf level, that means $n/4$ nodes may require only 1 shiftdown operation.
- From the remaining $n/4$ nodes that occur at heights between 0 to two levels up from leaves, $n/8$ nodes may be shifteddown by 2.
- From our observations about the maximum number of nodes at a level, we conclude only one node requires h shiftdowns.

In summary, total number of shiftdown operations is given by the expression:

$$(0.n/2) + (1.n/4) + (2.n/8) + \ldots + (h.1)$$

Rewriting the expression in terms of a summation, and simplifying we have:

$$\begin{aligned}
\sum_{1}^{h} \frac{n}{2^{k+1}} &= \frac{n}{4} \sum_{1}^{h} \frac{n}{2^{k-1}} \\
&< \frac{n}{4} \sum_{1}^{\infty} \frac{k}{2^{k-1}} \\
&= \frac{n}{4} \sum_{1}^{\infty} k x^{k-1}, \text{ put } x = \frac{1}{2} \\
&= \frac{n}{4} \frac{\mathrm{d}(\sum_{1}^{\infty} x^k)}{\mathrm{d}x} \\
&= \frac{n}{4} \frac{\mathrm{d}(\frac{1}{1-x})}{\mathrm{d}x} \\
&= \frac{n}{4} \frac{1}{(1-x)^2} \\
&= \frac{n}{4} \frac{1}{((1-1/2)^2)}, \text{ put back } x = \frac{1}{2} \\
&= n
\end{aligned}$$

The figure below illustrates building a heap highlighting the shiftdown operations.